SAFARITRAK

Massive Change Master Prompt

Phase 1: Project Foundation • Product Direction • Visual Reference Library • Implementation Rules

A consolidated execution brief for GitHub Copilot in VS Code

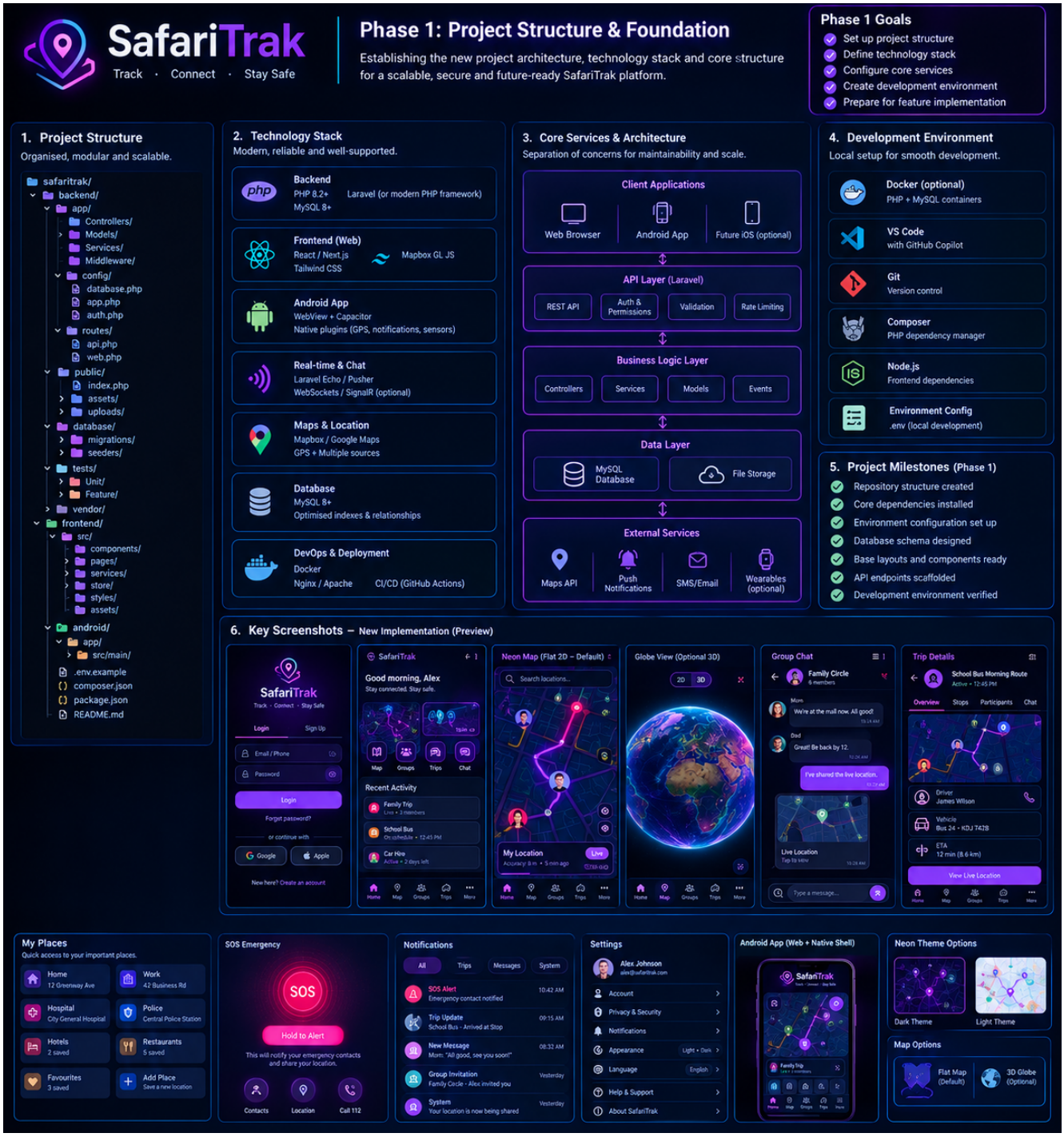
Purpose
Establish a clean, scalable foundation for the redesigned SafariTrak platform, then implement it in controlled phases using the supplied visual references. The screenshots are design targets—not static assets to place in the interface.

Important
This is a project-wide transformation brief. Copilot should inspect the existing repository only enough to identify reusable assets/data and migration risks, then establish the agreed new structure. Do not blindly preserve the old architecture, but do not destroy useful data or secrets.

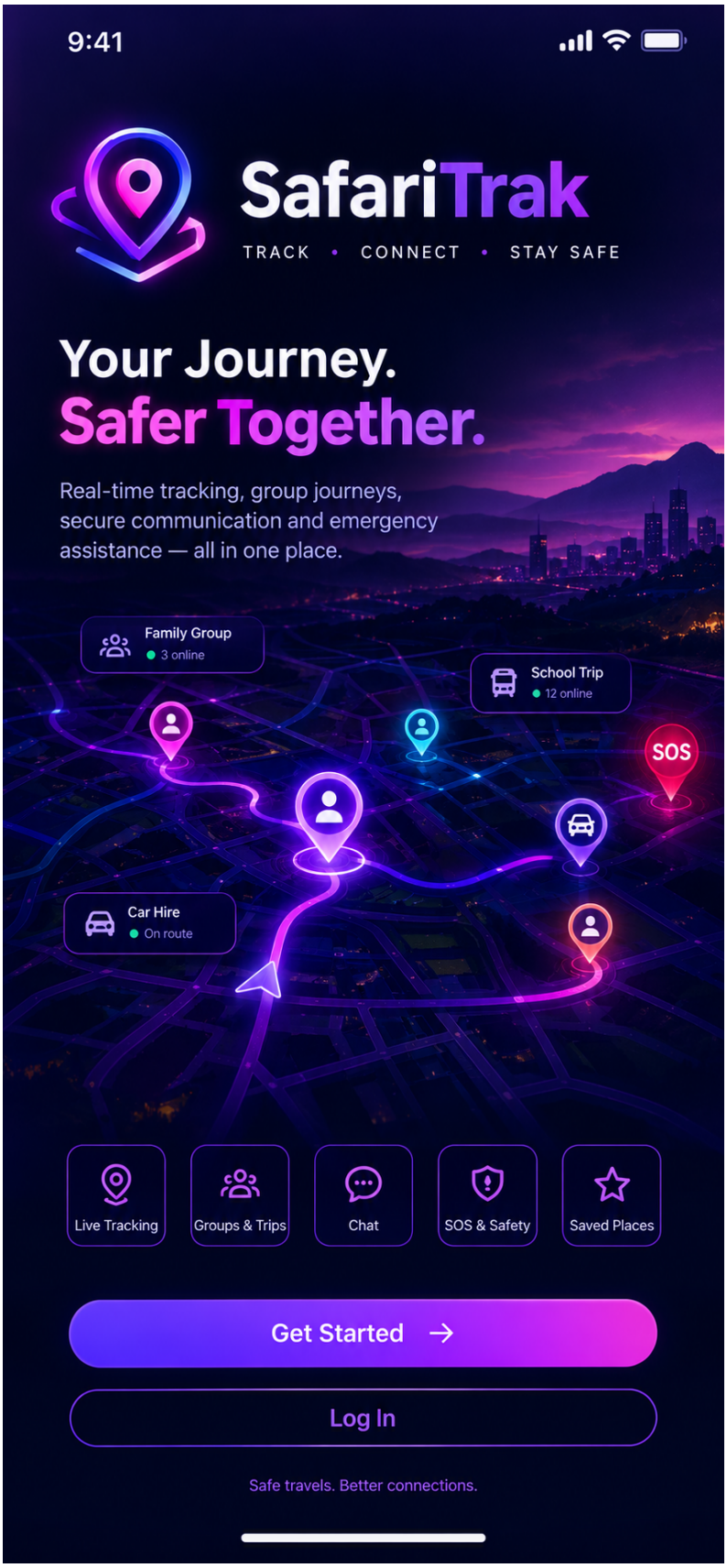
1. Visual references from this conversation

The following three generated images are included at readable size. Use them as a visual direction set. The collage is an overview only; create and implement each real screen separately.

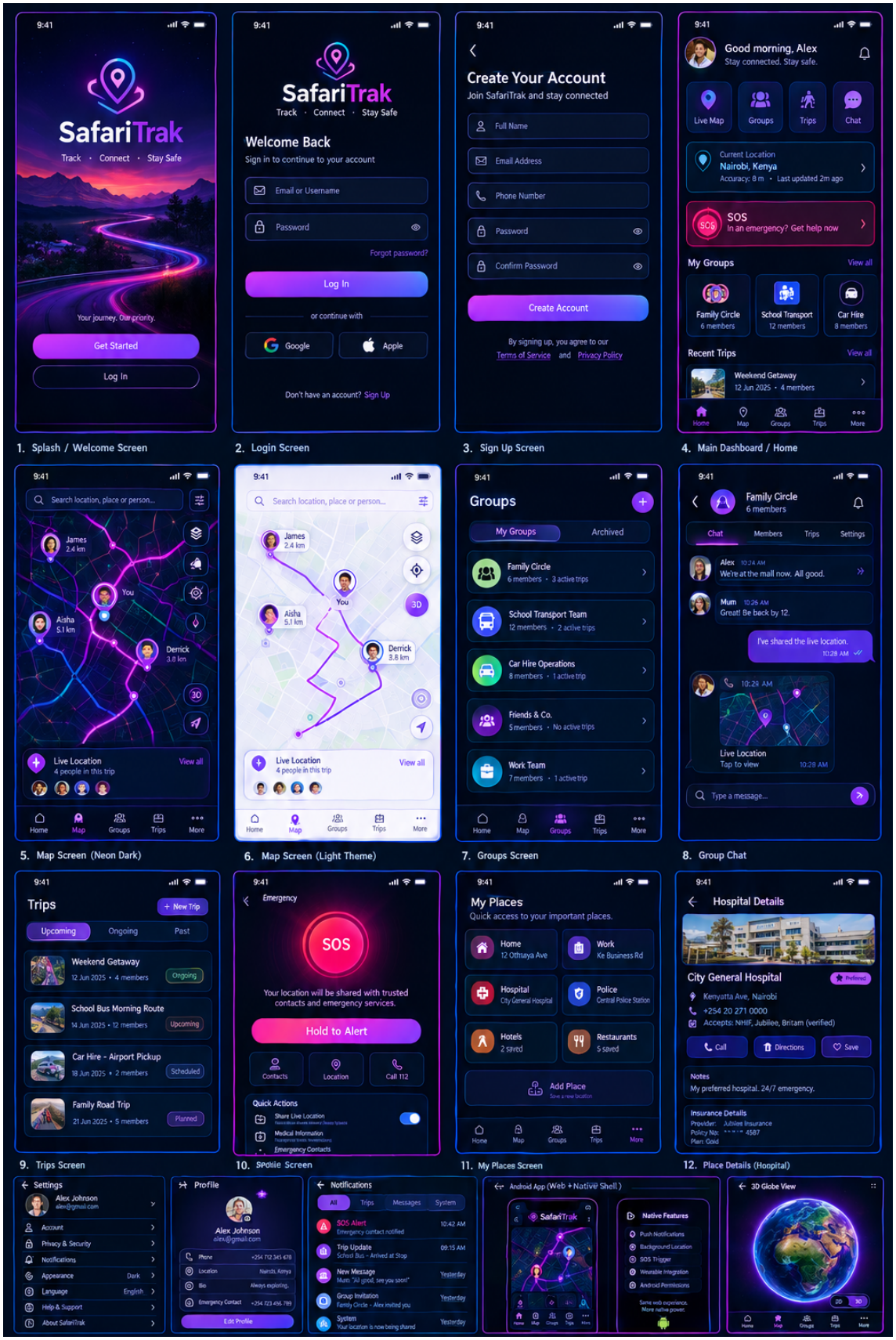
Reference 1 — SafariTrak Phase 1 project structure and architecture overview



Reference 2 — Welcome / onboarding visual direction



Reference 3 — Multi-screen UI overview (use as a guide; implement screens individually)



2. Master prompt — paste into GitHub Copilot Agent mode

Copy the full prompt below. Keep the three image references accessible in the project (for example, under docs/design-references/) and attach or open them for Copilot when possible. If Copilot cannot inspect an image directly, use the written requirements as authoritative.

```
ROLE
Act as the lead software architect, senior full-stack engineer, UX/UI designer, database designer, QA engineer, and Android integration planner for SafariTrak.

MISSION
Transform the existing SafariTrak project into a coherent, modern, modular tracking, journey coordination, communication, and safety platform. We expect substantial changes, so do not treat the old structure as sacred. First establish the new project foundation, then implement feature phases in order. Work in the existing repository and explain important decisions. Do not pretend a feature is complete unless it is implemented and tested.

WORKING RULES
1. Begin with Phase 1: structure and foundation. Do not attempt every product feature in one enormous edit.
2. Before major changes, inspect repository status and identify uncommitted work. Never overwrite user changes blindly.
3. Preserve useful existing assets and data where practical. Create a backup/export and a migration/rollback plan before destructive schema changes. Never delete production data.
4. Never hardcode credentials or expose secrets. Use documented environment variables and a safe .env.example with placeholders only.
5. Keep local setup reproducible and document exact Windows/XAMPP/VS Code commands where relevant.
6. Use small, reviewable changes; after each milestone run relevant tests, linters, build checks, and smoke tests. Fix regressions before continuing.
7. Do not use screenshots as static UI. Implement actual responsive pages, reusable components, functional controls, loading/empty/error states, validation, accessibility, and keyboard/focus behaviour.
8. Do not invent successful integrations. If a map provider, push service, email/SMS provider, wearable, or emergency-service API needs credentials, implement a clear adapter/interface and document setup.
9. Avoid adding paid dependencies/services without explaining the cost and offering a reasonable development alternative.
10. Record architectural decisions, assumptions, risks, and unfinished work in documentation.

PRODUCT VISION
SafariTrak: “Your journey. Safer together.” A shared platform for live location, trips, persistent groups, chat, SOS, saved places, and organisation-specific workflows. Do not lock the product into only schools, car hire, or personal tracking. Design a shared core with configurable roles and workflows.

DESIGN SYSTEM
- Brand: SafariTrak; tagline “Track • Connect • Stay Safe”.
- Visual direction: premium dark midnight surfaces with luminous purple/violet/magenta gradients and restrained cyan/turquoise accents. Red/pink is reserved for SOS and urgent danger. Do not make green the dominant brand colour.
- Provide both dark neon and light neon themes. Light mode must retain the purple/magenta identity and maintain strong text/road-label contrast.
- Default map is a practical flat 2D map. Offer an obvious “2D Map / 3D Globe” control; globe is optional and must not replace the everyday map.
- Use clean typography, consistent spacing, readable contrast, subtle glow, restrained glass effects, clear hierarchy, and responsive layouts. Avoid excessive glow that reduces legibility.
- Create reusable design tokens for colour, type, spacing, radii, elevation, borders, focus states, and motion. Respect reduced-motion preferences.
- Use the supplied images as inspiration, not pixel-perfect constraints where usability or platform behaviour conflicts.
- Desktop and mobile are first-class layouts. Keep touch targets large and support keyboard/focus navigation, including TV/remote-style directional navigation if feasible and relevant.
- Generate/store individual full-resolution reference screenshots, one screen per image, with descriptive names. The existing collage is only an index. Build a screen catalogue mapping image filename → route/screen → states → responsive variants → acceptance criteria.

PHASE 1 — PROJECT STRUCTURE AND FOUNDATION
A. Establish the target architecture before implementing feature-heavy screens.
B. Inspect current stack and repository; choose the least disruptive maintainable architecture compatible with the existing PHP/MySQL app and future Android client. Do not replace PHP/MySQL solely for fashion. If a backend refactor is needed, keep a documented reason and migration path.
C. Organise code by responsibility: frontend application, shared UI/design system, backend/API, domain/business logic, data access, database migrations/seeds, tests, docs/design references, scripts, and deployment/configuration. Adapt names to the actual chosen stack; do not create empty folders without purpose.
```

D. Establish clear configuration boundaries, environment-variable validation, consistent API response/error conventions, logging, request validation, authentication/authorization middleware, and database access patterns.
E. Decide how the web frontend and future Android app will use the same secure API. Document authentication/session/token strategy, CORS/CSRF implications, and native-only capabilities. Prefer a shared API and shared business rules rather than duplicating logic.
F. Create a database strategy with versioned migrations, explicit relationships, indexes, constraints, seed data for local development, and safe rollback guidance. Do not run destructive migrations against real data.
G. Add dependency scripts and documented commands for setup, development, tests, linting, production build, and database migration. Ensure commands work in the user's Windows environment; note PowerShell execution-policy quirks if relevant.
H. Create architecture documentation, directory tree, environment-variable reference, setup guide, migration/rollback guide, test plan, design system documentation, and a phased implementation roadmap.
I. Establish baseline quality gates: formatting/linting where appropriate, automated tests, frontend build, backend smoke tests, authentication tests, and basic accessibility checks.
J. Create the first reusable shell: app layout, navigation, page container, buttons, inputs, cards, dialogs/drawers, notifications/toasts, loading/empty/error states, theme switcher, and responsive breakpoints. Do not implement every complex feature in this phase.
K. Ensure there is a coherent working app shell with routes and placeholder states for planned screens, not dead buttons. Clearly label anything not yet implemented.
L. After Phase 1, report every file created/changed, architecture decisions, commands run and their actual results, remaining risks, and the next phase. Stop for review before proceeding to the next major phase unless the user explicitly asks to continue.

TARGET STRUCTURE — ADAPT TO THE ACTUAL STACK
Use a clear, documented structure along these lines, not necessarily these exact names:
- docs/architecture/
- docs/product/
- docs/design-references/
- docs/implementation/
- frontend/src/app/
- frontend/src/components/ui/
- frontend/src/components/layout/
- frontend/src/features/{auth,map,groups,trips,chat,safety,places,organisations,settings}/
- frontend/src/lib/
- backend/app/{Controllers,Services,Models,Middleware,Policies,Events}/ (if appropriate for PHP framework)
- backend/routes/
- backend/config/
- backend/database/{migrations,seeds}/
- backend/tests/{Unit,Feature}/
- scripts/
- tests/e2e/ (if suitable)
- .env.example
- README.md
- CHANGELOG.md or docs/implementation/progress.md

Do not force a React/Laravel/Capacitor migration before assessing current code and project constraints. If recommending a framework change, explain the concrete benefits, migration cost, risks, and how existing data/routes will be handled. Prefer incremental migration where possible.

CORE PRODUCT DOMAINS TO PLAN FOR
1. Accounts and identity: registration, login, recovery, sessions, profile, account deletion/export where appropriate.
2. Groups: permanent groups for family/friends or operations, membership, invitations, roles, permissions, persistent group chat, archive/delete controls. Groups remain until explicitly archived/deleted, but are not undeletable.
3. Trips: trips can be created from a persistent group; participants, route/stops, start/end, ETA, trip-specific location sharing, event history, upcoming/active/past states. Group membership alone must not silently enable continuous location tracking.
4. Live location: timestamp, freshness, accuracy, permission state, connectivity, battery where legitimately available, last-known position. Show stale/offline state clearly.
5. Multiple location sources: optional participant phones or dedicated tracker sources. Compare timestamps, accuracy, plausibility and freshness; flag disagreement for review. Do not claim sources are independent if they share a vehicle/device/GPS environment. App must still work with one source.
6. SOS and safety: immediate alert to designated contacts/coordinators; do not delay SOS while waiting for GPS corroboration. Include best available location and timestamp, delivery/acknowledgement/resolution status, retry/offline behaviour where possible. Never claim police/ambulance dispatch unless a verified integration confirms it.
7. Chat: persistent group chat, trip chat, authorised driver/customer chat, location sharing, announcements, message delivery states, mute/report/block controls and clear system-event styling. SOS must not depend on the chat screen.
8. Organisations: configurable workspaces and workflows for school transport, car hire/fleets, field

teams, and personal groups. Enforce role- and record-level permissions. School guardians must only see authorised children/trips.
9. My Places: saved home/work/school, hospitals, insurer-linked providers, police/ambulance/fire services, hotels/restaurants and custom favourites. Coverage must be labelled unverified unless checked against a trusted current insurer source. Medical data is optional and private.
10. Wearables/health devices: optional future integration via supported OS/vendor APIs. Separate explicit SOS, possible safety event and ordinary measurements. Never diagnose from a single measurement or trigger an emergency solely from an unconfirmed reading.
11. Maps: usable 2D default, optional 3D globe, search, participants, route layers, accessible markers, map legend, recenter and selected-location details. Keep map provider behind an adapter where practical.
12. Notifications and events: trip start/end, arrival, approaching stops, missed check-in, stale location, SOS, invitations and message alerts. Separate technical health events from user-reported danger.
13. Android: evaluate a native shell such as Capacitor only after documenting tradeoffs. Web UI may be reused, but GPS background behaviour, push, permissions, foreground services and wearable integrations need Android-specific implementation and testing. Do not promise unrestricted background tracking.
14. Admin and audit: role-based administrative tools, limited access, audit trails for sensitive location access and policy-based retention.

PRIVACY AND SAFETY REQUIREMENTS
- Clearly show when location is being shared and with whom; sharing must be scoped, consent-aware, revocable, and never silently enabled.
- Restrict location history access by role and purpose. Define retention/deletion policy.
- Protect minors' data with guardian authorisation and organisation-specific controls.
- Use least privilege, server-side authorisation on every protected record, secure password/session handling, validation, output escaping, rate limiting and safe error messages.
- Treat unusual GPS readings as a data-quality warning, not automatic proof of danger.
- Plan for low connectivity, permission denial, battery saving, stale positions and device shutdown.
- Keep emergency actions direct and accessible; never delay alerting to complete nonessential forms.

SCREEN AND IMAGE LIBRARY
Create an inventory and separate visual references for at least:
01 Welcome/splash
02 Login
03 Registration
04 Password recovery/reset
05 Main dashboard — mobile
06 Main dashboard — desktop
07 Flat 2D map — dark theme
08 Flat 2D map — light theme
09 Optional 3D globe
10 Map search/results/location details
11 Groups list
12 Create/edit group
13 Group details/members/roles
14 Persistent group chat
15 Trips list (upcoming/active/past)
16 Create trip and invite participants
17 Trip overview/route/stops
18 Active trip live tracking
19 Trip chat
20 SOS activation/hold-to-confirm state
21 SOS active/emergency details
22 Emergency contacts and safety preferences
23 Notifications centre
24 My Places overview
25 Add/edit saved place
26 Hospital details and coverage status
27 Connected devices/location sources
28 Profile
29 Account and privacy/security settings
30 Appearance/theme settings
31 Organisation/workspace switcher
32 School transport dashboard
33 School bus live tracking
34 Guardian/child trip view
35 Car-hire/fleet dashboard
36 Driver/customer trip communication
37 Admin/user and role management
38 Empty/loading/error/offline states
39 Desktop responsive navigation and map layout
40 Android shell/native-permission examples

For each screen, document:
- Screen name and image filename.
- User goal and entry point.
- Main components and hierarchy.
- All primary actions and what they do.
- Data required and loading/empty/error/offline states.
- Roles/permissions and privacy constraints.
- Mobile and desktop behaviour.
- Theme variants and accessibility requirements.
- Acceptance criteria and related tests.

Generate images one screen at a time at high enough resolution for the target device/aspect ratio. Do not rely on a collage to communicate fine detail. If image generation is unavailable, create the written screen specification and mark the reference as pending. Keep the images as documentation assets, not production UI assets.

IMPLEMENTATION ROADMAP AFTER PHASE 1
Plan and implement in dependency order, adjusting to evidence:
Phase 2: secure authentication, configuration, database foundation, API conventions and test infrastructure.
Phase 3: design system, app shell, routing, responsive layout, themes and screen catalogue.
Phase 4: groups, invitations, roles, permissions and persistent group chat foundation.
Phase 5: trips, participants, stops, live location and trip event model.
Phase 6: map UX, stale-location indicators and optional multiple source comparison.
Phase 7: SOS, emergency contacts, acknowledgements, notification reliability and offline edge cases.
Phase 8: saved places and trusted place/provider data.
Phase 9: organisation workspaces and school/car-hire workflows.
Phase 10: Android packaging/native integrations after web/API stability.
Phase 11: end-to-end security, performance, accessibility, backup/restore, deployment and release readiness.
Do not blindly follow this order if the dependency analysis demonstrates a better one; document changes.

ACCEPTANCE CRITERIA FOR PHASE 1
- A documented target architecture and repository structure exist.
- Local setup instructions are reproducible on Windows.
- Environment configuration uses placeholders and does not leak secrets.
- Database migration and rollback strategy is documented.
- The web app launches into a coherent responsive shell with working routes.
- Shared design tokens and dark/light theme foundation exist.
- Planned screens have a route/screen catalogue and individual reference-image mapping.
- Lint/build/tests or documented smoke checks run, with truthful results.
- No major existing user data is destructively altered.
- A report lists files changed, commands/results, unresolved decisions and recommended next phase.

FINAL RESPONSE AFTER WORK
Summarise:
1. What was actually completed.
2. Key files created/changed.
3. Commands/tests run and real results.
4. How to run locally.
5. Any migration, security, or deployment warnings.
6. Which screen references are available and which remain pending.
7. The next recommended phase.
Do not say “done” for anything that was only planned or scaffolded.

3. Screen-by-screen visual reference plan

Generate and save one screen per image, not one collage. Use descriptive filenames in docs/design-references/ and link each image from the screen catalogue.

Screen set	Contents	Purpose
01–04	Welcome, login, registration, password recovery	Auth and onboarding
05–06	Mobile and desktop dashboards	Primary home experience
07–10	Dark 2D map, light 2D map, 3D globe, search/details	Mapping and location
11–14	Groups list, group editor, group details, group chat	Persistent groups
15–19	Trips list, create trip, trip overview, active tracking, trip chat	Journey workflows
20–22	SOS active, emergency details, emergency contacts	Safety
23–27	Notifications, places, add place, hospital details, devices	Supporting utilities
28–30	Profile, privacy/security, appearance	Account and preferences
31–37	Workspace switcher, school dashboards, guardian view, fleet, driver chat, admin	Organisation workflows
38–40	Offline/error states, desktop navigation, Android/native permissions	Resilience and platform support

Use the images as a design guide, not as evidence that features exist. For each reference, pair the visual with written behaviour and acceptance criteria.

4. Practical execution sequence

• Create a Git checkpoint/branch before the transformation and confirm the working tree is understood.

• Add the generated reference images and this PDF to project documentation, then create the screen catalogue.

• Have Copilot complete Phase 1 only; review architecture, structure, setup and the app shell before continuing.

• Implement one coherent vertical slice at a time, including database/API/UI/tests, rather than building every screen as a disconnected mockup.

• After each phase, run the app locally and verify the key flows before asking Copilot to proceed.

End of master brief